

The ASP.NET Core Form Tag Helpers Cheat Sheet

Published on Jan 22, 2019

How to get data from your HTML forms to ASP.NET Core (using Tag Helpers).

- The Form (MVC)
- The Form (Razor Pages)
- Labels
- Text Inputs
- Passwords
- TextAreas
- Dropdown Lists (Select)
- Hidden Inputs
- Checkboxes
- Radio Buttons

The Form itself (MVC)

CSHTML #

```
@model FormsCheatSheet.Models.Example
```

```
<form asp-controller="Example" asp-action="Save" >
    <button type="submit">Submit Form</button>
</form>
```

The Controller

```
public class ExampleController : Controller
{
    public IActionResult Index()
    {
        return View(new Example());
    }

    [ValidateAntiForgeryToken]
    public IActionResult Save(Example model)
    {
        // do something with model e.g. model.Name
        return Ok();
    }
}
```

Resulting HTML

```
<form action="/Example/Save" method="post">
    <input name="__RequestVerificationToken" type="hidden"
    value="<random-looking-value=here>">
</form>
```

Using default ASP.NET Core MVC routing, a request to view `https://<your-app-here>/example` will invoke the `Index` method which returns `Example\Index.cshtml` (including our form).

When that form is submitted, the data will be routed to `Save` and the model populated with any submitted values.

The `__RequestVerificationToken` is automatically generated by ASP.NET and included as a hidden field. When the form is submitted this will be included in the form data and ASP.NET will validate it to ensure this is a legitimate request.

This helps ensure the form isn't some spoof form someone has created on a different web site, pointing to your application.

The Form itself (Razor Pages)

CSHTML

```
@model FormsCheatSheet.Models.Example

<form asp-page="About">
    <button type="submit">Submit Form</button>
</form>
```

The Razor Page (About.cshtml.cs)

```
public class AboutModel : PageModel
{
    public void OnGet()
    {
    }

    public void OnPost()
    {
        // form will submit to here
    }
}
```

Resulting HTML

```
<form action="/About" method="post">  
  <input name="__RequestVerificationToken" type="hidden"  
    value="<random-looking-value=here>">  
</form>
```

Here we explicitly inform ASP.NET that this form will post back to the About page.

ASP.NET Core will then attempt to locate and invoke an `OnPost` method in the About page's Page Model (`About.cshtml.cs`) to handle the submitted form data.

Legacy .NET web apps causing you grief?

Build modern, reliable web applications, faster with .NET and Blazor.

Build better .NET web apps

Label

CSHTML

```
<label asp-for="Name"></label>
```

Model

```
[Display(Name = "Full Name")]  
public string Name { get; set; }
```

Resulting HTML

```
<label for="Name">Full Name</label>
```

If you provide a `[Display(Name="")]` its value will be used for the label, otherwise you'll just get the name of the property itself e.g. `Name`.

Text Input

CSHTML

```
<input asp-for="Name" />
```

Model

```
public string Name { get; set; }
```

Resulting HTML

```
<input type="text" id="Name" name="Name" value="">
```

ASP.NET Core will infer that this should be an input of type `text` because the `'Name'` property on the model is a string.

When this form is submitted the `Name` property on the model will be populated using the value entered in the form.

Password Input

CSHTML

```
<input asp-for="Password" />
```

Model

```
[DataType(DataType.Password)]  
public string Password { get; set; }
```

Resulting HTML

```
<input type="password" id="Password" name="Password">
```

ASP.NET Core will infer that this should be an input of type `password` because of the `[DataType]` attribute.

Alternatively you can just specify this yourself in the CSHTML markup e.g. `<input asp-for="Password" type="password"/>`.

When this form is submitted, `Password` will be populated using the value entered in the form.

Textarea

CSHTML

```
<textarea asp-for="Bio" />
```

Model

```
public string Bio { get; set; }
```

Resulting HTML

```
<textarea id="Bio" name="Bio"></textarea>
```

When submitted, Bio will be populated using the value entered.

Dropdown list (Select)

CSHTML

```
<select asp-for="Country" asp-items="Model.Countries"></select>
```

Model

```
public List<SelectListItem> Countries { get; set; }  
    = new List<SelectListItem>  
    {  
        new SelectListItem("UK", "UK"),  
        new SelectListItem("USA", "USA"),  
        new SelectListItem("France", "FR")  
    }; // used to populate the list of options  
  
public string Country { get; set; }
```

Resulting HTML

```
<select id="Country" name="Country">
```

Jon Hilton - Making sense of ASP.NET



```
    <option value="FR">France</option>
</select>
```

The `select` tag helper needs to know where to fetch its list of options from.

Unlike the `asp-for` attribute (which assumes everything you try to use is on the Model), you have to explicitly tell `asp-items` where the list is coming from. In this case, `Model.Countries`.

When the form is submitted `Country` will be set to the selected value in the dropdown list.

Hidden Field

CSHTML

```
<input asp-for="ApplicationType" />
```

Model

```
[HiddenInput]
public string ApplicationType { get; set; } = "Online";
```

Resulting HTML

```
<input type="hidden" id="ApplicationType" name="ApplicationType" value="Online">
```

Occasionally you may need to include a value in the submitted form data but not show it on the page.

You can use hidden fields for this, setting them to a value which will then be posted back when the form is submitted.

The `[HiddenInput]` attribute means that ASP.NET will infer this should be a hidden field.

Alternatively you can set this yourself in the CSHTML.

```
<input type="hidden" asp-for="ApplicationType">
```

In this example the `ApplicationType` of 'Online' will be sent "up" to the client and rendered as a hidden input. When the form is subsequently submitted this value will be included in the submitted form data and the `ApplicationType` property on the model will be set accordingly.

Checkbox

CSHTML

```
<input asp-for="AcceptsTerms" />
```

Model

```
public bool AcceptsTerms { get; set; }
```

Resulting HTML

```
<input type="checkbox" data-val="true" id="AcceptsTerms"  
data-val-required="The AcceptsTerms field is required."/>
```

```
name="AcceptsTerms" value="true">
```

ASP.NET Core will infer that the input should be of type "checkbox" if you point it at a Boolean property on your model.

The `value` attribute here is a little confusing as it seems to always be `true`. However, there is another attribute which isn't visible in the rendered markup called `checked` and this is the important one which will change depending on whether the checkbox is ticked or not.

When the form is submitted the posted form data will include a `true` or `false` and the `AcceptTerms` property on the model will be populated accordingly.

Because the boolean property on the model cannot be null (has to be either true or false) ASP.NET also generates validation attributes here indicating that this input is required.

This is largely irrelevant though as, even if your users don't touch the checkbox, a `false` value will be posted when the form is submitted and the Model property will be set accordingly.

Radio Button

CSHTML

```
<label><input asp-for="Gender" value="M" type="radio" />Male</label>  
<label><input asp-for="Gender" value="F" type="radio"/>Female</label>
```

Model

```
public string Gender { get; set; }
```

Resulting HTML

```
<label><input value="M" type="radio" id="Gender" name="Gender">Male</label>  
<label><input value="F" type="radio" id="Gender" name="Gender">Female</label>
```

Radio Buttons are different to checkboxes in that they generally come in groups of two or more and you can only select one of the possible options.

In this scenario the user can select Male or Female and the `Gender` property on the model will be set accordingly.

Legacy .NET web apps causing you grief?

Build modern, reliable web applications, faster with .NET and Blazor.

Build better .NET web apps

Next Up

1. [Adding tons of code without understanding why...](#)
2. [What to focus on when learning ASP.NET Core?](#)
3. [MVC vs Razor Pages - A quick comparison](#)

© 2025 Jon Hilton. All rights reserved.



[Contact](#) [Privacy Policy](#) [Cookie Policy](#)